1. CUSTOMER (CID, CName, City, LoyaltyLevel)

CID	CName	City	LoyaltyLevel
C1	Alice	San Jose	Gold
C2	Bob	Fremont	Silver
C3	Carol	Oakland	Bronze
C4	David	San Jose	Gold
C5	Emma	Sunnyvale	Silver

2. PRODUCT (PID, PName, Category, Price)

PID	PName	Category	Price
P1	Laptop	Electronics	1200
P2	Phone	Electronics	800
P3	Chair	Furniture	150
P4	Table	Furniture	400
P5	Headphones	Electronics	100

1.
Cross Join : Customer x Product

CID	CName	City	Loyalty Level	PID	PName	Category	Price
C1	Alice	San Jose	Gold	P1	Laptop	Electronics	1200
C1	Alice	San Jose	Gold	P2	Phone	Electronics	800

C1	Alice	San Jose	Gold	P3	Chair	Furniture	150
C1	Alice	San Jose	Gold	P4	Table	Furniture	400
C1	Alice	San Jose	Gold	P5	Headphones	Electronics	100
C2	Bob	Fremont	Silver	P1	Laptop	Electronics	1200
C2	Bob	Fremont	Silver	P2	Phone	Electronics	800
C2	Bob	Fremont	Silver	P3	Chair	Furniture	150
C2	Bob	Fremont	Silver	P4	Table	Furniture	400
C2	Bob	Fremont	Silver	P5	Headphones	Electronics	100
C3	Carol	Oakland	Bronze	P1	Laptop	Electronics	1200

C3	Carol	Oakland	Bronze	P2	Phone	Electronics	800
C3	Carol	Oakland	Bronze	P3	Chair	Furniture	150
C3	Carol	Oakland	Bronze	P4	Table	Furniture	400
C3	Carol	Oakland	Bronze	P5	Headphones	Electronics	100
C4	David	San Jose	Gold	P1	Laptop	Electronics	1200
C4	David	San Jose	Gold	P2	Phone	Electronics	800
C4	David	San Jose	Gold	P3	Chair	Furniture	150
C4	David	San Jose	Gold	P4	Table	Furniture	400
C4	David	San Jose	Gold	P5	Headphones	Electronics	100

C5	Emma	Sunnyvale	Silver	P1	Laptop	Electronics	1200
C5	Emma	Sunnyvale	Silver	P2	Phone	Electronics	800
C5	Emma	Sunnyvale	Silver	P3	Chair	Furniture	150
C5	Emma	Sunnyvale	Silver	P4	Table	Furniture	400
C5	Emma	Sunnyvale	Silver	P5	Headphones	Electronics	100

The cross join combines every row from the CUSTOMER table with every row from the PRODUCT table.. This operation is also known as a Cartesian product, resulting in a total of 25 rows (5 customers × 5 products). The result does not reflect any existing relationship between the data, as it simply pairs all customers with all products. It is used to create all possible combinations between two tables for eg in color and size combination in clothing for food and drinks combination in restaurants.

2. EQUIIJOIN : CUSTOMER⋈CUSTOMER.CID=ORDER.CIDORDER

CID	CName	City	LoyaltyLevel	OID	OrderDate	TotalAmount
C1	Alice	San Jose	Gold	O1	2025-08-01	1300
C2	Bob	Fremont	Silver	O2	2025-08-03	950
C3	Carol	Oakland	Bronze	O3	2025-08-05	150
C4	David	San Jose	Gold	O4	2025-08-06	1600
C1	Alice	San Jose	Gold	O5	2025-08-08	100

The equijoin successfully links customers to their orders by matching on the CID column, which acts as the primary key in the CUSTOMER table and a foreign key in the ORDER table. Unlike a cross join, this operation is meaningful because it only includes rows where a relationship exists, providing a clear list of who placed which orders. In our table, Alice (CID C1) appears twice because she has placed two separate orders (O1 and O5).

3.

Natural Join : ORDER ⋈ ORDER_ITEM

OID	CID	OrderDate	TotalAmount	PID	Quantity
O1	C1	2025-08-01	1300	P1	1
O1	C1	2025-08-01	1300	P5	1
O2	C2	2025-08-03	950	P2	1
O2	C2	2025-08-03	950	P5	1
O3	C3	2025-08-05	150	P3	1
O4	C4	2025-08-06	1600	P1	1
O4	C4	2025-08-06	1600	P4	2
O5	C1	2025-08-08	100	P5	1

This operation combines the information from both tables based on a common column, showing the order details alongside the specific items included in each order. The combined table will have 8 rows, as there are 8 line items in the ORDER_ITEM table. The natural join is effective for combining tables with a common attribute. In this case, it links the parent order (ORDER table) to its specific components (ORDER_ITEM table). This provides a more detailed view than either table alone, enabling you to see, for example, that order O4 had two different products (P1 and P4) and that Alice's order (O1) contained a laptop and headphones.

4. Left Semi Join : CUSTOMER ⋉ ORDER

CID	CName	City	LoyaltyLevel
C1	Alice	San Jose	Gold
C2	Bob	Fremont	Silver
C3	Carol	Oakland	Bronze
C4	David	San Jose	Gold

The left semi-join is used to determine which rows in the left table (**CUSTOMER**) have a match in the right table (**ORDER**), without including any columns from the right table. The result shows that four customers (**Alice**, **Bob**, **Carol**, and **David**) have placed at least one order. **Emma** (CID C5) is not included because her CID does not appear in the **ORDER** table. This operation is useful for filtering a set of records based on the existence of related records in another table.

5. Right Semi Join : ORDER_ITEM ⋉ PRODUCT

PID	PName	Category	Price
P1	Laptop	Electronics	1200
P2	Phone	Electronics	800
P3	Chair	Furniture	150
P4	Table	Furniture	400
P5	Headphones	Electronics	100

This operation returns all rows from the **PRODUCT** relation that have a matching PID in the **ORDER_ITEM** relation. The right semi-join is used to find rows in the right table (**PRODUCT**) that have a match in the left table (**ORDER_ITEM**). The result shows that all five products have appeared in at least one order. The products that have not been ordered would not appear in this result. This operation is useful for filtering a set of records based on the existence of related records in another table.

6. Left Outer Join : CUSTOMER ⟕ ORDER

CID	CName	City	LoyaltyLevel	OID	OrderDate	TotalAmount
C1	Alice	San Jose	Gold	O1	2025-08-01	1300
C2	Bob	Fremont	Silver	O2	2025-08-03	950
C3	Carol	Oakland	Bronze	O3	2025-08-05	150
C4	David	San Jose	Gold	O4	2025-08-06	1600
C1	Alice	San Jose	Gold	O5	2025-08-08	100
C5	Emma	Sunnyvale	Silver	NULL	NULL	NULL

The left outer join provides a complete list of all customers. The key aspect of this join is the inclusion of **Emma** (CID C5), who does not have any corresponding entries in the **ORDER** table. Her order-related columns are filled with NULLvalues, signifying that she has not placed any orders. This operation is essential when you need a full list of records from one table and want to see if they have any related data in another table.

7. RIGHT OUTER JOIN : ORDER ⟖ PRODUCT

OID	PID	Quantity	PName	Category	Price
O1	P1	1	Laptop	Electronics	1200
O4	P1	1	Laptop	Electronics	1200
O2	P2	1	Phone	Electronics	800
O3	P3	1	Chair	Furniture	150
O4	P4	2	Table	Furniture	400
O1	P5	1	Headphones	Electronics	100
O2	P5	1	Headphones	Electronics	100
O5	P5	1	Headphones	Electronics	100

The right outer join ensures that every product is listed, regardless of whether it has been sold. If a product had existed in the PRODUCT table but not in any orders, it would still appear in this result, with NULL values in the OID and Quantity columns. This operation is essential when you need a comprehensive list from the "right" table and want to see if they have any related data in the "left" table.

8. UNION :

For electronics:

Table1 = ΠCID(σCategory='Electronics'(CUSTOMER⋈ORDER⋈ORDER_ITEM⋈PRODUCT))

For furnitures:

Table2 = ΠCID(σCategory='Furniture'(CUSTOMER⋈ORDER⋈ORDER_ITEM⋈PRODUCT))

finalTable = Table1 U Table 2

CID
C1
C2
C3
C4

The union operation effectively merges the distinct CIDs from both sets of customers. By finding the set of customers who ordered Electronics and the set of customers who ordered Furniture, and then performing a union, we get a complete list of all customers who ordered either type of product. The union automatically removes duplicates, providing a clean list of unique customer IDs.

9. Intersection :
$CID\_Electronics = \Pi CID(\sigma Category = 'Electronics'(CUSTOMER \bowtie ORDER \bowtie ORDER\_ITEM \bowtie PRODUCT))$
$CID\_Furniture = \Pi CID(\sigma Category = 'Furniture'(CUSTOMER \bowtie ORDER \bowtie ORDER\_ITEM \bowtie PRODUCT))$

$CID\_Electronics \cap CID\_Furniture$

CID
C4

The intersection operation finds the common elements between two sets. In this case, we're looking for customer IDs that are present in both the set of customers who bought Electronics and the set of customers who bought Furniture. The result shows that only customer C4 (David) satisfies this condition. This is because David's order (O4) includes a Laptop (Electronics) and a Table (Furniture).

10. Minus :
$CID\_Furniture = \Pi CID(\sigma Category = 'Furniture'(CUSTOMER \bowtie ORDER \bowtie ORDER\_ITEM \bowtie PRODUCT))$
$CID\_Electronics = \Pi CID(\sigma Category = 'Electronics'(CUSTOMER \bowtie ORDER \bowtie ORDER\_ITEM \bowtie PRODUCT))$

$CID\_Furniture - CID\_Electronics$

CID
C3

The minus operation finds the customer IDs present in the first set (Furniture customers) but not in the second set (Electronics customers). The result shows that only customer C3 (Carol) ordered furniture but not electronics. Customers C1 and C4 are excluded from the result because, although they ordered furniture, they also ordered electronics.